Software Test Plan (STP)

Campus Ride-Share Matching

Version: 1.0

Authors: Keertika, Khushi, Tanishkaa , Pavithra

Date: 27 October 2026

1. Introduction

Purpose

This plan defines the scope, approach, environment, responsibilities, and exit criteria for testing Campus Ride-Share Matching.

Scope

Testing covers account access, ride offers and requests, location fields, geospatial search, route-compatibility matching, match display, in-app messaging/notifications, validation, authorization, and basic performance. Third-party map services and production infrastructure are excluded unless integrated into the project.

References

Campus Ride-Share Matching SRS; Software Architecture and Design Specification (SAD); project implementation and matching rules.

Definitions

SRS: Software Requirements Specification. SAD: Software Architecture and Design. STP: Software Test Plan. FR: Functional Requirement. NFR: Non-Functional Requirement.

2. Test Items

Modules

Authentication/access control; ride offer management; ride request management; location/geospatial search; route matching and ranking; match status; in-app messaging/notifications; input validation; persistence layer.

3. Features to be Tested

Functional coverage

TC-01: Valid and invalid sign-in/access. TC-02: Create/edit/cancel a ride offer. TC-03: Create/edit/cancel a ride request. TC-04: Required-field and invalid-value validation. TC-05: Geospatial search returns posts within configured bounds. TC-06: Matching heuristic identifies and ranks compatible routes. TC-07: Cancelled/expired posts are excluded. TC-08: Match details show correct locations and time. TC-09: Send/read a message in the correct conversation. TC-10: Relevant match/message notification appears if implemented.

Non-functional coverage

TC-11: Search response-time measurement against the agreed test dataset. TC-12: Authorization checks prevent users editing another user's post. TC-13: Error handling for unavailable database/service. TC-14: Basic responsive layout and usability check. TC-15: Confirm secrets and sensitive credentials are not written to logs or committed to the repository.

4. Features Not to be Tested

Exclusions

External map-provider accuracy, third-party notification delivery, real-world ride safety, payment processing, and production-scale availability are excluded unless explicitly implemented and approved for the project.

5. Test Approach / Strategy

Levels

Unit testing for validation and matching functions; integration testing for API/database and messaging interactions; system testing for complete student workflows; acceptance testing against the project requirements.

Types

Functional, negative, integration, regression, basic performance, authorization/security, and usability testing.

Entry criteria

A runnable build is available; test accounts and sample ride data are prepared; required services and database are configured; requirements and matching rules are reviewed.

Exit criteria

All planned high-priority test cases are executed; no unresolved critical defects remain; failed tests are documented; requirements coverage is reviewed; and the team approves known limitations.

6. Test Environment

Environment

Use the team's agreed local development environment or test deployment, a test database, at least two test student accounts, and synthetic ride offers/requests with known pickup/drop-off coordinates and times.

Tools and data

Use the project's existing test framework where available. API tools or browser developer tools may be used for integration checks. Prepare positive, negative, nearby-location, distant-location, overlapping-route, incompatible-route, cancelled-post, and invalid-input test data.

7. Test Schedule

Proposed sequence

1. Review requirements and prepare test cases. 2. Prepare environment and test data. 3. Execute unit and integration tests.
4. Execute end-to-end workflows. 5. Run regression and basic performance checks. 6. Fix defects, retest, and prepare a test summary. Dates should be filled in by the team based on the course deadline.

8. Test Deliverables

Deliverables

This test plan; test cases/checklist; synthetic test data; test execution results; defect list; requirements-to-test traceability; and a final test summary.

9. Roles and Responsibilities

Roles

Developer(s): implement fixes and support tests. Tester/QA owner: prepare cases, execute tests, record results and defects. Project coordinator: confirm requirements, resolve scope questions, and approve submission. Replace role labels with actual team member names before submission.

10. Risks and Mitigation

Risk: inconsistent test locations

Mitigation: use a documented synthetic dataset with known coordinates and expected matches.

Risk: matching rules are ambiguous

Mitigation: document route-compatibility scoring and thresholds before validating results.

Risk: external services are unavailable

Mitigation: use local test data or mocks where appropriate.

Risk: late changes break existing flows

Mitigation: rerun core regression tests after changes.

11. Assumptions and Dependencies

Assumptions

The project brief and core matching rules are available; the application can be run in a test environment; test accounts and synthetic ride records can be created; and any optional external services are identified by the team.

12. Suspension and Resumption Criteria

Suspend testing if

The application cannot start, the test database is unavailable, or a blocking defect prevents execution of a significant portion of planned cases.

Resume testing when

The blocking defect is fixed, the environment is stable, and the affected smoke tests pass.

13. Test Case Management and Traceability

Traceability examples

FR-04 create offer → TC-02. FR-07 create request → TC-03. FR-11 geospatial search → TC-05. FR-12/FR-13 matching/ranking → TC-06. FR-15 exclude cancelled posts → TC-07. FR-16 messaging → TC-09. NFR-01 response time → TC-11. NFR-03 authorization → TC-12.

14. Test Metrics and Reporting

Metrics

Number and percentage of test cases executed; pass/fail/blocked counts; defects by severity; retest status; requirement coverage; and observed search response time.

Reports

Maintain a test execution checklist and a concise final summary of passed tests, failed tests, open defects, and limitations.

15. Approvals

Approval fields

Project coordinator: _____ Date: _____. Developer/QA representative: _____ Date: _____.

Software Test Plan - Execution Results

Geospatial Search and Filtering | Version 1.1 | 03 November 2026

Scope and environment

This addendum records Khushi's geospatial verification for FR-10, FR-11, NFR-01, TC-05, and TC-11. Tests ran locally against the application HTTP API, MongoDB database **campus-ride-share-geospatial-integration-test**, and the matching-engine rank endpoint. The test database is disposable and is removed after execution.

Test outcomes

Case	Result	Execution evidence
TC-05 FR-10 / FR-11	**PASS**	Authenticated offer/request flow and spatial search passed. Valid GeoJSON coordinate order, origin and destination radius, departure window, and matching-engine payload were verified. A route point about 0.99 km from the origin was included at a 1 km radius; wrong-destination, out-of-radius, and out-of-window offers were excluded. Invalid ranges, null/blank coordinates, zero and over-limit radii were rejected. A valid no-match search returned count 0.
TC-11 NFR-01	**PASS**	Benchmark used 1,005 active offer records in the disposable local test database, three warm-up searches, and 25 timed HTTP searches. p95: **23.54 ms**; maximum: **25.99 ms**. Both are below the 3,000 ms requirement. This is a local test result, not a production load guarantee.

Deterministic synthetic route coordinates

These fixtures are centralized in **test-data/campus-routes.json** for repeatable integration tests. They are test data, not a production campus-route seed catalog.

Fixture	Latitude	Longitude	Use
PES Main Gate	12.9716	77.5946	Search origin and offer/request origin
Electronic City	12.8399	77.6770	Destination used for the matching route
Near-radius boundary	12.9805	77.5946	Approximately 0.99 km from search origin

Additional edge cases

Offer creation rejects an out-of-range latitude and null coordinate values. Search rejects null or blank coordinates, incomplete or invalid input, and radius values outside the configured (0, 100] km range. Empty valid searches return HTTP 200 with zero offers. The integration test drops its dedicated database during cleanup.